

Dependency Inversion Principle

The D in SOLID, explained with a payment processing example

High-level modules should not depend on low-level modules. Both should depend on abstractions. The word worth unpacking is "inversion," because it is not obvious what gets inverted.

The problem

```
class PaymentService {
    private CreditCardPayment payment = new CreditCardPayment();

    void checkout(Money total) {
        payment.pay(total);
    }
}
```

`PaymentService` is your business logic. `CreditCardPayment` is a detail that talks to a card gateway. Right now the business logic depends on the detail.

Three things go wrong:

- Adding UPI means editing `PaymentService`. It probably grows an if-else on payment type.
- You cannot test it. Every test run hits a real card gateway, because `PaymentService` creates its own dependency and you cannot get in between.
- When the gateway vendor changes, that change reaches into your business logic.

The fix

```
interface PaymentMethod {
    void pay(Money amount);
}

class PaymentService {
    private final PaymentMethod payment;

    PaymentService(PaymentMethod payment) {
        this.payment = payment;
    }

    void checkout(Money total) {
        payment.pay(total);
    }
}
```

```
new PaymentService(new CreditCardPayment()).checkout(total);
new PaymentService(new UpiPayment()).checkout(total);
new PaymentService(new FakePayment()).checkout(total);    // tests
```

`PaymentService` no longer knows card gateways exist.

What got inverted

Before: `PaymentService` → `CreditCardPayment`
After: `PaymentService` → `PaymentMethod` ← `CreditCardPayment`

Before, the arrow pointed down: high-level depending on low-level. After, both point at the interface. The low-level class now depends on something the high-level side defined. That reversal is the inversion.

The detail that makes it real: `PaymentMethod` belongs to `PaymentService`, not to the payment implementations. It lives in your domain package and is written in your business vocabulary. If you put it in the gateway package and shape it around what the card SDK happens to offer, you have added an interface without inverting anything, and the next payment method will not fit.

One clarification

DIP and dependency injection get used interchangeably, and they are not the same thing.

Injection is the mechanic: pass the dependency in rather than constructing it. DIP is the design decision: which direction does the dependency point, and who owns the abstraction. You can inject a concrete `CreditCardPayment` into the constructor and satisfy injection while still violating DIP.

Where it pays off

The same pattern applies to anything at the edge of your system. `OrderService` depending on `OrderRepository` rather than `PostgresOrderRepository`. `NotificationService` depending on `Notifier` rather than `SmtpClient`. In each case the business logic defines what it needs, and the infrastructure conforms.

Why it matters most: DIP is the principle that makes the other five practical. Open/Closed needs an abstraction to extend. Liskov needs a contract to substitute against. DIP is what puts that contract in the right place.